

实验三：基于 LSTM 的唐诗自动续写

朱立景 2025E8013282177

一、概述

本实验旨在构建一个能够根据用户给定首句自动续写唐诗的字符级语言模型。给定任意一句诗作为开头（如「湖光秋月两相和」），模型应续写出语义连贯、符合汉语表达习惯的后继诗句。

任务定义： 将诗歌续写建模为字符级序列预测问题——给定前缀字符序列，逐字预测下一个汉字，直至生成结束符 `<EOP>`。

数据集： 预处理后的唐诗数据集 `tang.npz`，共 57,580 首唐诗，词表大小 8,293，每首诗最大长度 125 个 token。

解决方案： 在实验要求的基础组件（Embedding、LSTM、全连接层）之上，设计增强版 `PoetryModel`：3 层 LSTM 提取局部序列特征，叠加因果自注意力捕获长程依赖，双层全连接头完成下一字分类；采用交叉熵损失进行下一字预测训练，配合 AdamW 优化器与余弦学习率衰减；推理阶段使用 temperature + top-k 采样生成诗句。

运行环境： Conda, Python 3.10, PyTorch 2.5.1 + CUDA, NVIDIA A800 GPU。

二、解决方案

2.1 网络结构设计

网络整体数据流为：

```
1  字符索引 → Embedding(512) → 3层LSTM(768) → LayerNorm
2      → 因果Self-Attention(+残差) → LayerNorm → Dropout
3      → FC(768→512) → ReLU → FC(512→vocab) → 下一字 logits
```

模型总参数量约 **2,406 万**。在保留实验要求核心组件的同时，通过注意力机制与残差连接增强对诗歌对仗、句读和长程语义关联的建模能力。

核心代码 — 因果自注意力与 PoetryModel：

```
1  class CausalSelfAttention(nn.Module):
2      def __init__(self, dim: int):
3          super().__init__()
4          self.q_proj = nn.Linear(dim, dim)
5          self.k_proj = nn.Linear(dim, dim)
6          self.v_proj = nn.Linear(dim, dim)
7          self.scale = dim ** -0.5
8
9      def forward(self, x, key_padding_mask=None):
10         q, k, v = self.q_proj(x), self.k_proj(x), self.v_proj(x)
11         scores = torch.matmul(q, k.transpose(-2, -1)) * self.scale
12         causal_mask = torch.triu(torch.ones(x.size(1), x.size(1), device=x.device,
13 dtype=torch.bool), diagonal=1)
14         scores = scores.masked_fill(causal_mask.unsqueeze(0), float("-inf"))
15         if key_padding_mask is not None:
16             scores = scores.masked_fill(key_padding_mask.unsqueeze(1), float("-inf"))
17         attn = torch.softmax(scores, dim=-1)
18         return torch.matmul(torch.nan_to_num(attn, nan=0.0), v)
19
20 class PoetryModel(nn.Module):
21     def __init__(self, vocab_size, emb_dim=512, hidden_dim=768, num_layers=3):
```

```

21         super().__init__()
22         self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=PAD_IDX)
23         self.lstm = nn.LSTM(emb_dim, hidden_dim, num_layers, batch_first=True, dropout=0.3)
24         self.ln1 = nn.LayerNorm(hidden_dim)
25         self.attn = CausalSelfAttention(hidden_dim)
26         self.ln2 = nn.LayerNorm(hidden_dim)
27         self.dropout = nn.Dropout(0.5)
28         self.fc1 = nn.Linear(hidden_dim, 512)
29         self.fc2 = nn.Linear(512, vocab_size)
30
31     def forward(self, x, hidden=None):
32         emb = self.embedding(x)
33         lstm_out, hidden = self.lstm(emb, hidden)
34         h = self.ln1(lstm_out)
35         h = self.ln2(h + self.attn(h, key_padding_mask=(x == PAD_IDX)))
36         logits = self.fc2(F.relu(self.fc1(self.dropout(h))))
37         return logits, hidden

```

设计要点：

组件	作用
Embedding(512)	将 8,293 个汉字/符号映射为稠密向量
3 层 LSTM(768)	捕获字符间的局部顺序模式与韵律结构
因果 Self-Attention	在自回归约束下，让当前字关注全部前文上下文
LayerNorm + 残差	稳定深层网络训练，缓解梯度消失
双层 FC 头	非线性映射至词表，输出下一字概率分布
padding mask	注意力计算中屏蔽 <code></s></code> 填充位，避免无效信息干扰

推理阶段续写逻辑：

```

1 def generate_poetry(model, prefix, word2ix, ix2word, device, temperature=0.8, top_k=5):
2     tokens = [START_IDX] + [word2ix[c] for c in prefix if c in word2ix]
3     for _ in range(max_len - len(tokens)):
4         logits, _ = model(torch.tensor([tokens], device=device))
5         next_id = sample_next_token(logits[0, -1], temperature, top_k) # top-k 采样
6         tokens.append(next_id)
7         if next_id == EOP_IDX:
8             break
9     return decode_tokens(tokens, ix2word)

```

2.2 损失函数设计

采用交叉熵损失（CrossEntropyLoss）进行下一字预测（Language Modeling）：

- 输入/目标构造： `input = batch[:, :-1]`， `target = batch[:, 1:]`，即每个位置预测下一个字符。
- 忽略填充： `ignore_index=8292`（`</s>`），不对填充位置计算损失。
- 标签平滑： `label_smoothing=0.1`，防止模型过度自信，使生成文本更流畅。

```
1 criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX, label_smoothing=0.1)
2
3 def _compute_lm_loss(logits, targets, criterion):
4     b, t, v = logits.shape
5     return criterion(logits.reshape(b * t, v), targets.reshape(b * t))
```

评价指标：除 loss 外，以困惑度（Perplexity, PPL）衡量语言模型质量：`PPL = exp(val_loss)`，越低表示模型对测试数据的预测越准确。

2.3 优化器设计

配置项	取值	说明
优化器	AdamW	带权重衰减的自适应优化，适合大规模参数
学习率	1e-3	初始学习率
权重衰减	1e-4	L2 正则，抑制过拟合
学习率调度	CosineAnnealingLR	30 epoch 内余弦退火，平滑降低学习率
梯度裁剪	max_norm=5.0	防止 LSTM 训练中梯度爆炸
Batch Size	128	平衡训练速度与显存占用
Epochs	30	约 15 分钟/epoch（GPU）

```
1 optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
2 scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30)
3
4 # 训练循环中
5 loss.backward()
6 nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
7 optimizer.step()
8 scheduler.step()
```

2.4 创新点

在实验基本要求（Embedding + LSTM + FC）之上，本实验做了以下增强：

- 因果自注意力 + 残差连接**：LSTM 擅长建模局部顺序，但对长距离对仗、意象呼应的捕获有限；叠加因果注意力后，每个字可“回看”全部前文，同时残差连接保留 LSTM 的原始表征，二者互补。
- 左填充数据对齐**：原始 `tang.npz` 中诗句以左填充（`</s>`）方式存储，直接训练会导致注意力在填充区域产生 NaN。设计 `_left_align()` 将序列统一对齐为 `<START>诗句<EOP></s>...` 格式，是保障训练稳定的关键预处理步骤。
- temperature + top-k 采样**：推理时不采用贪心解码（易导致重复），而是在概率最高的 top-5 候选字中按温度系数 0.8 随机采样，兼顾确定性与多样性，生成更具诗意的文本。

三、实验分析

3.1 数据集介绍

属性	值
文件名	tang.npz
诗数量	57,580 首
词表大小	8,293
序列最大长度	125
特殊 token	<START> (8291)、<EOP> (8290)、</s> (8292)
数据划分	训练集 51,822 / 验证集 5,758 (9:1, seed=42)

每首诗的存储格式为 <START>诗句正文<EOP>，后跟 </s> 填充至固定长度。诗句为字符级序列，包含汉字及标点（，。！？等）。

3.2 实验结果

训练环境：NVIDIA A800 GPU（CUDA）Conda 环境。

训练过程（部分 epoch）：

Epoch	Train Loss	Val Loss	Perplexity	耗时
1	6.5837	6.4010	602.45	31.1s
5	5.4625	5.3898	219.17	30.5s
10	5.0201	5.1386	170.48	30.9s
15	4.7825	5.0835	161.34	30.8s
18	4.6748	5.0810	160.93	31.2s
30	4.4696	5.1078	165.31	42.2s

- 最佳验证 loss：5.0810（第 18 epoch）
- 最佳困惑度：160.93
- 总参数量：24,056,677
- 总训练时间：约 16 分钟（30 epoch）

损失曲线分析：

- 前 10 个 epoch loss 快速下降（PPL 从 602 降至 170），模型迅速学到基本的字符共现规律与标点用法。
- 第 10–18 epoch 进入平台期，验证 loss 缓慢优化至最优。
- 第 18 epoch 后验证 loss 略有回升，出现轻微过拟合迹象；可进一步采用早停（Early Stopping）在第 18 epoch 截止训练。

续写测试结果：

给定首句	模型续写结果	原诗（参考）
湖光秋月两相和	湖光秋月两相和，一曲清歌一曲歌。不似玉郎无限意，一声啼处到春波。	湖光秋月两相和，潭面无风镜未磨。遥望洞庭山水翠，白银盘里一青螺。
春眠不觉晓	春眠不觉晓，日暮又相思。风动秋风夜，风吹落玉枝。	春眠不觉晓，处处闻啼鸟。夜来风雨声，花落知多少。
床前明月光	床前明月光，池上秋风起。独自见秋光，空余落花处。	床前明月光，疑是地上霜。举头望明月，低头思故乡。

结果分析：

- 1. **格式正确性：** 模型能正确在首句后添加逗号、句号等标点，并生成五言或七言句式，基本符合唐诗格律。
- 2. **语义连贯性：** 续写内容在意象上与前文有一定关联（如「湖光秋月」后出现「清歌」「春波」），但整体语义与原诗差异较大，属于合理创作而非记忆复述。
- 3. **不足：** 存在个别重复用词（如「一曲清歌一曲歌」），以及意象堆砌但逻辑衔接不够紧密的情况。这主要受限于字符级建模难以显式学习对仗规则，且 PPL=160 仍有较大优化空间。
- 4. **改进方向：** 可增加训练 epoch 并配合早停；引入格律/韵脚约束；尝试更大规模预训练或 Transformer 架构；降低 temperature 提升确定性。

四、总结

本实验成功实现了基于深度学习的唐诗自动续写系统。在 57,580 首唐诗数据集上，采用 Embedding + 3 层 LSTM + 因果自注意力 + 双层全连接层的增强架构，以字符级语言模型方式训练，最终验证困惑度降至 **160.93**。模型能够根据用户输入的首句自动生成格式基本正确、语义基本连贯的后续诗句。

实验过程中，针对数据集左填充格式导致注意力层 NaN 的问题，设计了左对齐预处理方案，体现了对数据与模型协同设计的重视。推理阶段采用 temperature + top-k 采样策略，在多样性与流畅性之间取得平衡。

总体而言，本实验完成了「数据读取 → 网络构建 → 模型训练 → 续写测试」的完整流程，验证了循环神经网络在中文诗歌生成任务上的可行性，也为后续引入更复杂的生成策略（如藏头诗、格律约束）奠定了基础。